自然语言处理

第7讲：大语言模型预训练

杨沐昀

哈尔滨工业大学计算学部
语言技术研究中心

2026年秋季学期

yangmuyun@hit.edu.cn

主要内容

- 整体框架
- 预训练模型架构
- 分布式训练
- 预训练数据

大语言模型整体框架

- 预训练模型架构：扩大规模（Scale up）
- 三阶段训练流程： 拓展能力（Scale up）
  - 预训练(Pretraining)
  - 有监督精调(SFT)
  - 人类反馈强化学习(RLHF)



预训练

- 使用海量无标签数据
  - 让模型掌握“通用知识”
    - 学习语言结构、语法、语义与知识
    - 让模型具有强大的迁移学习能力
- 存在不足：
  - 对特定任务缺乏针对性

有监督精调

- 使用标注数据集
  - 让模型的输出符合任务的要求
  - 例如：
    - 在翻译任务中，能生成流畅且准确的译文
    - 在情感分析中，能对文本的情感倾向正确分类
- 存在不足：
  - 可能不符合人类偏好

人类反馈的强化学习

- 通过人类反馈引导模型优化
  - 与人类的选择和判断对齐
  - 减少有害、不准确或偏颇内容的生成概率
- RLHF训练流程：
  1. 使用已精调的模型生成一组候选输出。
  2. 人类标注人员根据候选输出的质量对其进行排序（如从最优到最差）。
  3. 利用排序数据训练奖励模型，区分模型输出的质量。
  4. 基于强化学习算法调整模型参数,生成更符合奖励模型期望的输出。

主要内容

- 整体框架
- 预训练模型架构
- 分布式训练
- 预训练数据

当前架构

- 参数数化状态空间模型（State Space Model, SSM）
  - 参数化的方法建模输入信号
  - 灵活性和通用性尚存不足
- 基于Transformer的架构（主流架构）
  - 自注意力机制（Self-Attention）
  - 多头注意力机制（Multi-Head Attention）
  - ...
  - 本课程重点介绍基于Transformer的架构
- 自回归模型（Autoregressive Model）
  - Decoder-only
  - OpenAI的GPT系列，META的LLaMA系列

补充：GPT模型架构发展历程

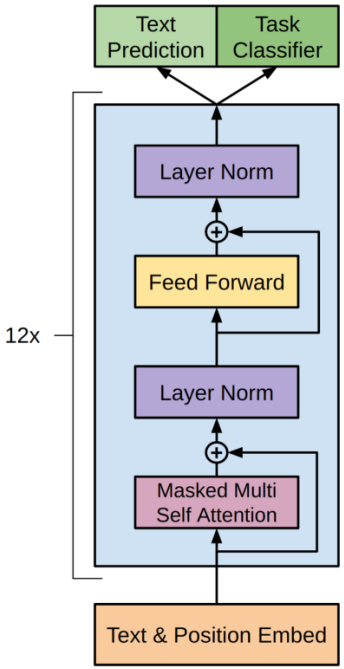
- 发展历程
  - GPT1 Decoder-only预训练采用生成式任务 （117M）
  - GPT2 沿用GPT1架构，小改动（归一化层以及残差层）训练方式上不需要先pre-train再finetune（124~150M）
  - GPT3 沿用GPT2，但大幅度增加参数 （175B）

补充：GPT模型架构发展历程

- GPT1的架构
  - 自监督预训练，对于 U = {u_1, ..., u_n}

$$L_1(\mathcal{U}) = \sum_i \log P(u_i|u_{i-k}, \ldots, u_{i-1}; \Theta)$$

  - 模型结构
    - 多层 Transformer 解码器
    - 对输入上下文标记应用多头自注意力操作
    - 前馈层
    - 归一化层
  - 有监督微调(用不同的下游任务进行微调)
    - 分类任务
    - 文本蕴含
    - 文本相似性
    - 问答和常识推理



补充：GPT模型架构发展历程

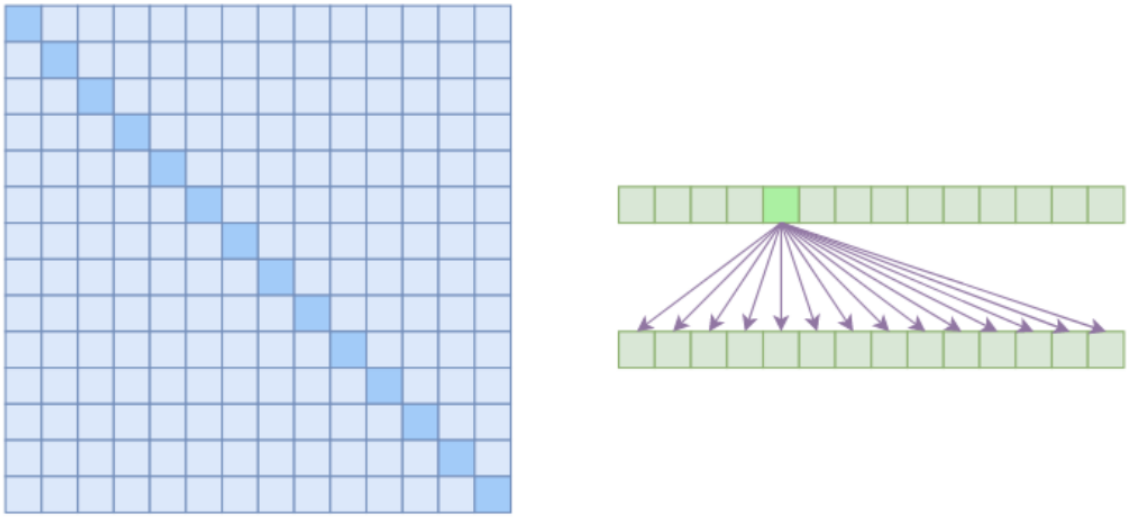
- GPT2的架构
  - 解决GPT1需要对每个下游任务进行微调导致的工作量过大问题
    - 希望只需要训练一个模型即可完成诸多NLP任务，实现zero-shot learning的效果
    - 训练过程舍弃了需要消耗大量资源的微调阶段
  - 主体架构和GPT1基本一致，差异在于：
    - Layer Norm被提前至Decoder Layer的输入位置，即对输入进行归一化
    - 残差层参数的初始化参数统一除以 $1/\sqrt{N}$， N 代表残差网络的层数
    - 词典大小变为50257，句子的长度从512变为1024，batch size变为512

补充：GPT模型架构发展历程

- GPT2的改进
  - GPT2与GPT1最主要的区别在于取消了二阶段训练的模型
  - 通过更加海量的数据和更多的模型参数训练出一个强大的单阶段模型，该模型能在一定程度上完成zero-shot learning任务

补充：GPT模型架构发展历程

- GPT3的架构变化
  - Sparse Transformer的结构
    - 标准Self-attention

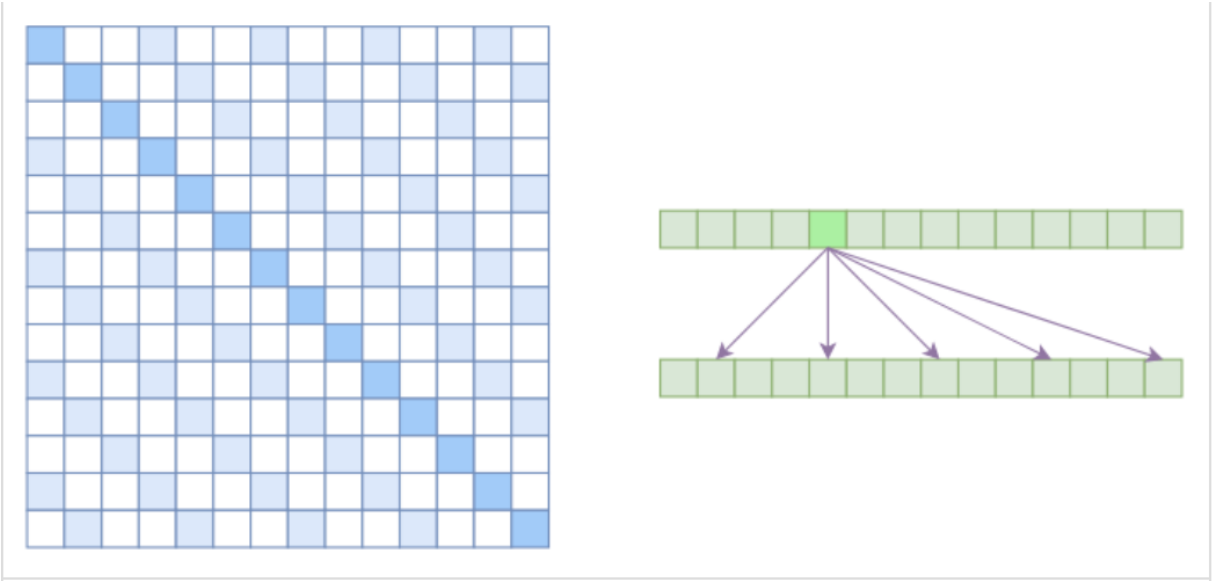


标准Self Attention的注意力矩阵（左）和关联图示（右）

https://blog.csdn.net/m0_37531129

补充：GPT模型架构发展历程

- GPT3的架构变化
  - Sparse Transformer的结构
    - 膨胀自注意力**Atrous Self Attention**

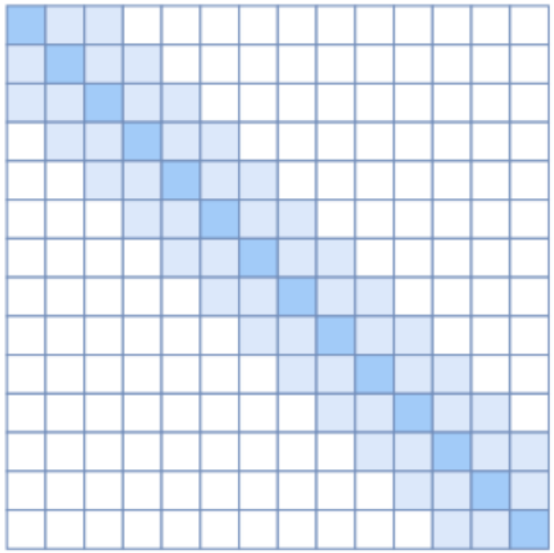


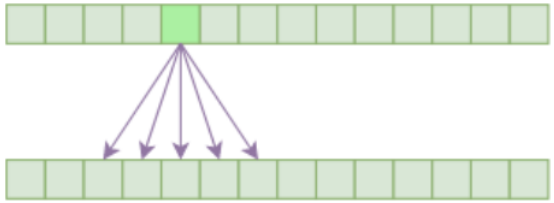
Atrous Self Attention的注意力矩阵（左）和关联图示（右）

https://blog.csdn.net/m0_37531129

补充：GPT模型架构发展历程

- GPT3的架构变化
  - Sparse Transformer的结构
    - 局部自注意力**Local Self Attention**



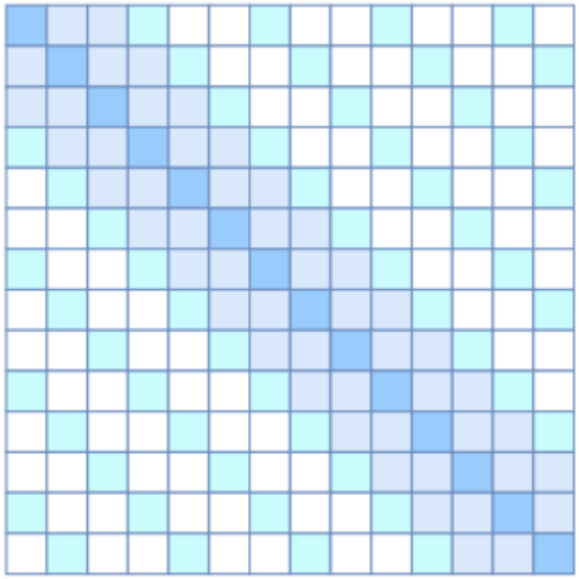


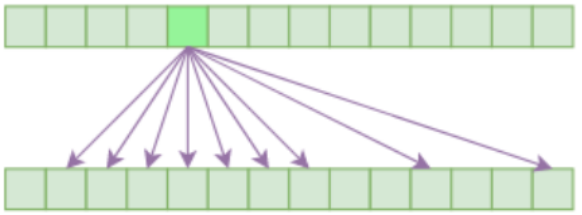
Local Self Attention的注意力矩阵（左）和关联图示（右）

https://blog.csdn.net/m0_37531129

补充：GPT模型架构发展历程

- GPT3的架构变化
  - Sparse Transformer的结构





Sparse Self Attention的注意力矩阵（左）和关联图示（右）

补充：GPT模型架构发展历程

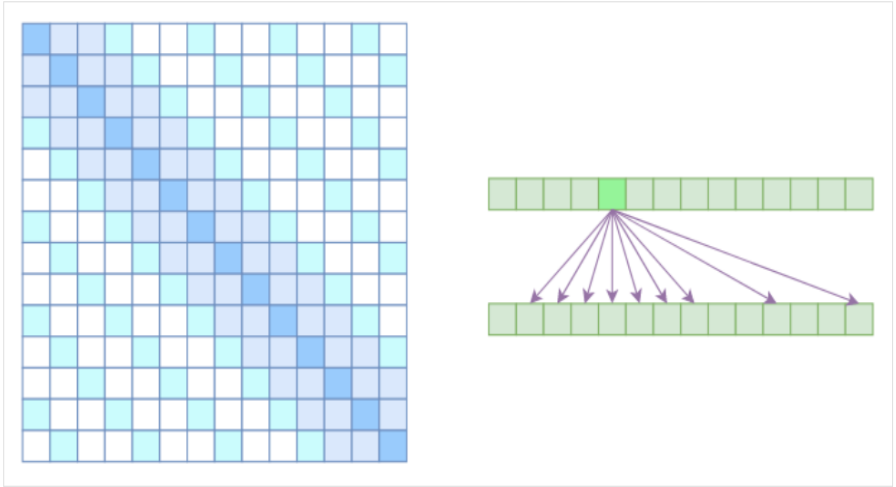
- GPT3的架构
  - 参数量的上升

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

  - 训练了8种参数量不同的模型，最大参数量为175B

补充：ChatGPT架构:参考GPT3

- GPT3的架构
  - 基本采用GPT2的架构
  - 96层Transformer
  - 每层96Head
  - 词嵌入12288
  - 文本长度2048
- Transformer采用的是类似Sparse Transformer的结构，以节省模型训练过程中的显存占用



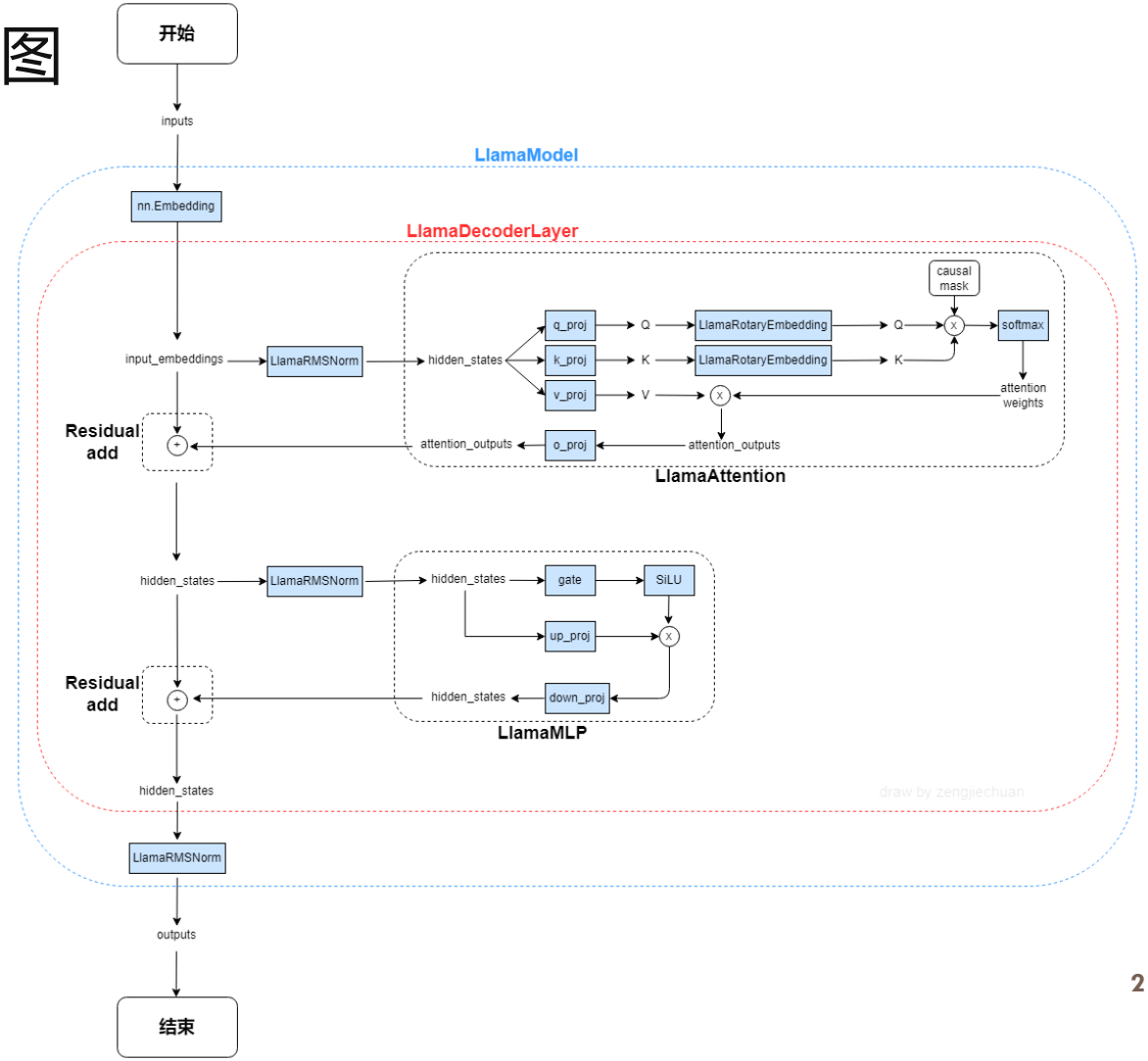
Sparse Self Attention的注意力矩阵（左）和关联图示（右）

补充：LLaMA的模型架构

- Decoder结构，结合了GPT3、 PALM等模型的优化方法
  - Pre-normalization预标准化：使用GPT3的方法，在每层的输入加归一化，归一化函数采用RMSNorm
  - SwiGLU 激活函数：参考PaLM，使用SwiGLU 激活函数替换了ReLU激活函数
  - Rotary Embeddings：参考GPTNeo，移除了绝对位置嵌入，使用了旋转位置嵌入(RoPE)

补充：LLaMA的模型架构

- Llama模型结构图



预训练模型架构：大模型的探索

- 激活函数
  - 变化有限
- 位置编码
  - 补充词向量中缺失的位置信息
- 注意力机制
  - 挖掘词元及更长句子成分（如短语、句子等）之间的相互关系

位置编码

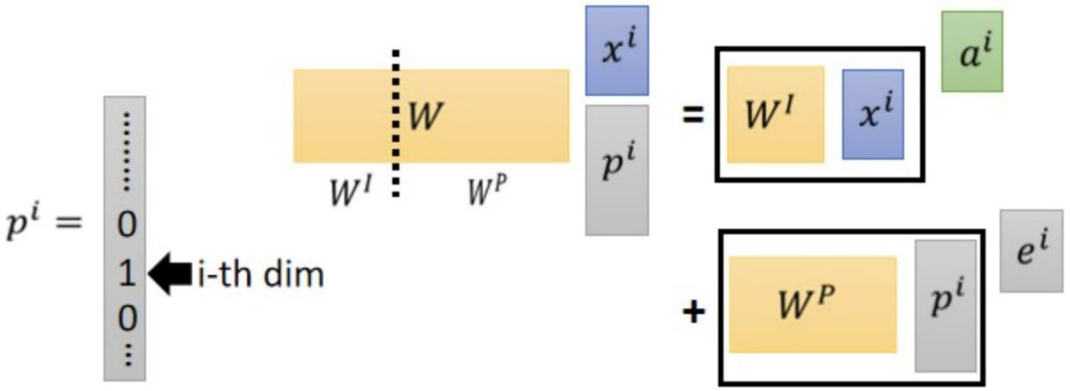
- 目的：引入位置信息
  - 弥补相对RNN的缺失
- Transfromer最初的实现
  - 直接将一个整数（位置索引值）映射为一个向量

$$\mathrm{PosEnc}(p,i)=\begin{cases}\sin\left(\frac{p}{10000^{\frac{i}{d}}}\right) & \text{if } i \text{ is even}\\ \cos\left(\frac{p}{10000^{\frac{i-1}{d}}}\right) & \text{else}\end{cases}$$

  - 可区分性：模型可以区分不同位置的元素。
  - 相对位置关系：可以通过位置编码的相加或相减来很容易地捕捉到元素之间的相对位置
  - 无限序列支持：能够处理可变长度的输入序列
  - 连续性和平滑性：输入序列中相邻位置的编码也是相似的；

位置编码

- 可学习（Learnable）的位置编码
  - 反向传播学习
  - 随训练过程更新



- 问题：
  - 长度固定，对长序列有限制性

位置编码

- 相对位置编码
  - 绝对位置编码的问题：不同位置的相同子句内部编码不一致
- 基本思想：
  - 一个词对另一个词的注意力计算，只取决于它们的相对位置关系，而与它们的绝对位置无关

$$\text{score}_{i,j} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} = \frac{\left(\mathbf{x}_i \mathbf{W}^Q\right) \cdot \left(\mathbf{x}_j \mathbf{W}^K + a^K_{i-j}\right)}{\sqrt{d_k}}$$

$$\mathbf{z}_i = \sum_{j=1}^{n} \alpha_{i,j} \cdot \mathbf{v}_j = \sum_{j=1}^{n} \alpha_{i,j} \cdot \left(\mathbf{x}_j \mathbf{W}^V + a^V_{i-j}\right)$$

位置编码

- 旋转位置编码（Rotary Position Embedding, RoPE）
  - 将向量引入复数域
  - 利用复数旋转的几何性

$$\mathbf{R}^d_{\mathbf{\Theta},m} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{\frac{d}{2}} & -\sin m\theta_{\frac{d}{2}} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{\frac{d}{2}} & \cos m\theta_{\frac{d}{2}} \end{pmatrix}$$

$$\mathbf{q}_m^T \mathbf{k}_n = \left(\mathbf{R}^d_{\Theta,m} \mathbf{W}_q \mathbf{x}_m\right)^T \left(\mathbf{R}^d_{\Theta,n} \mathbf{W}_k \mathbf{x}_n\right) = \mathbf{x}^T \mathbf{W}_q R^d_{\Theta,n-m} \mathbf{W}_k \mathbf{x}_n$$

层归一化（Layer Normalization，LN）

- 归一化的目的
  - 增加神经网络训练的稳定性
- 常见的层归一化方法
  - 批次归一化（Batch Normalization）
  - 均方根归一化（Root Mean Square Normalization，RMSNorm）
  - 深度归一化（DeepNorm）
  - …
- 本课程重点介绍均方根归一化（RMSNorm）
  - 具有轻量化和高效性的优点

层归一化

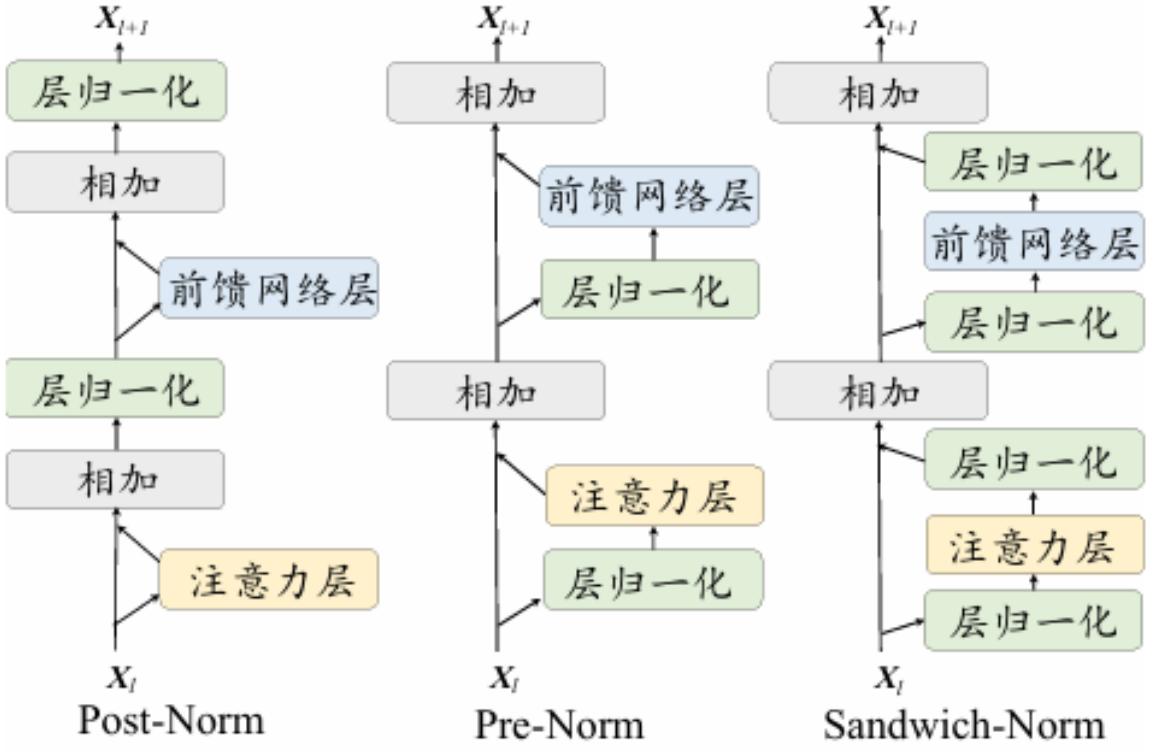
- 均方根归一化（RMSNorm）
  - 实验表明，使用RMSNorm归一化方法比起普通的批次归一化方法在训练速度和性能上均有一定提高。
  - 主要在于避免了期望和方差的计算

$$RMSNorm(x) = \frac{x}{RMS(x)} \times \gamma$$

$$RMS(x) = \sqrt{\frac{1}{H}\sum_{1}^{H} x_i^2}$$

层归一化的位置

- 层归一化的位置
  - 层后归一化
  - 层前归一化
  - 夹心归一化

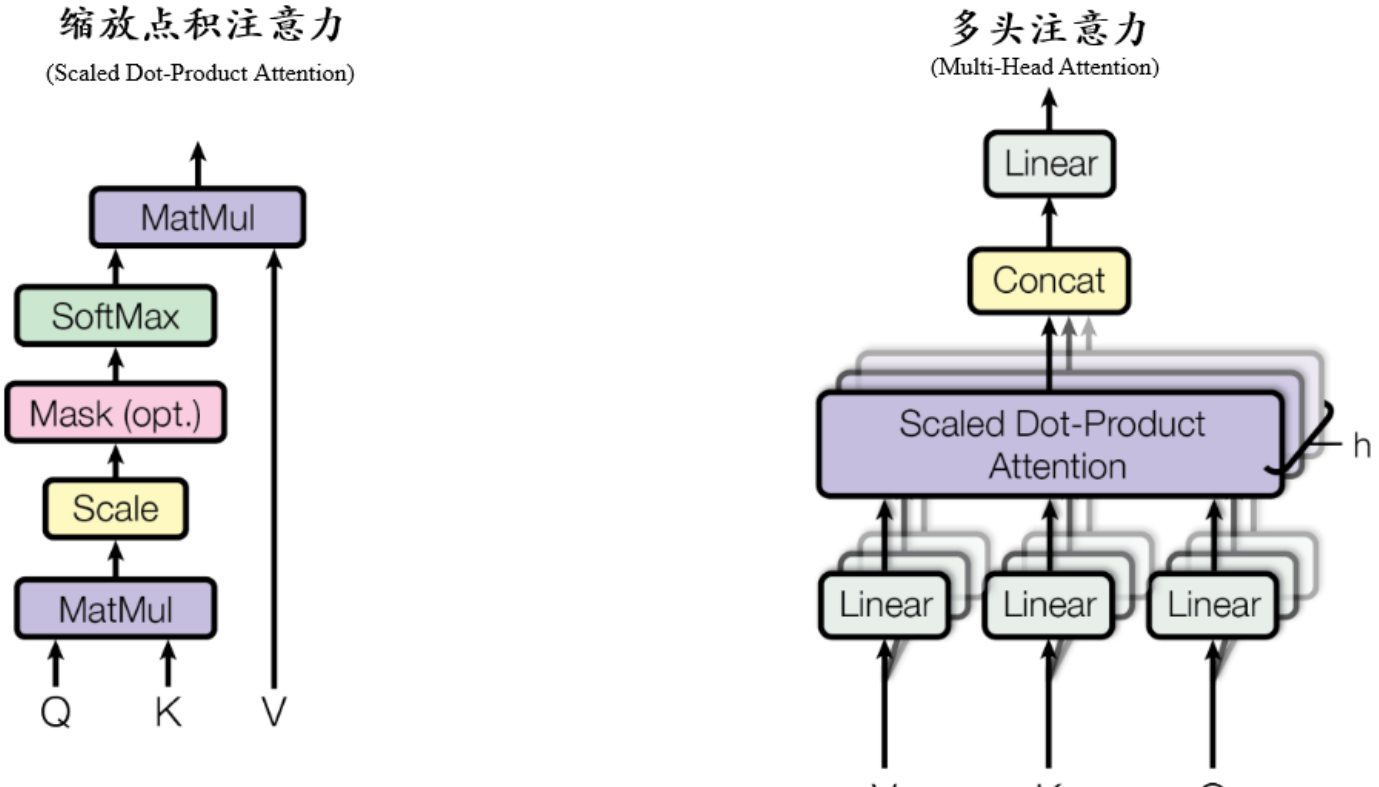


注意力机制

- 多头注意力机制（Multi-Head Attention，MHA）
  - Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017, 30.
- 多查询注意力（Multi-Query Attention，MQA）
  - Shazeer N. Fast transformer decoding: One write-head is all you need[J]. arXiv preprint arXiv:1911.02150, 2019.
  - 降低显存消耗和推理成本
- 群查询注意力（Group-Query Attention，GQA）
  - Ainslie J, Lee-Thorp J, De Jong M, et al. Gqa: Training generalized multi-query transformer models from multi-head checkpoints[J]. arXiv preprint arXiv:2305.13245, 2023.
  - 作为MHA和MQA之间的折中方案

多头注意力机制（MHA）

- MHA由多个独立的单头注意力构成



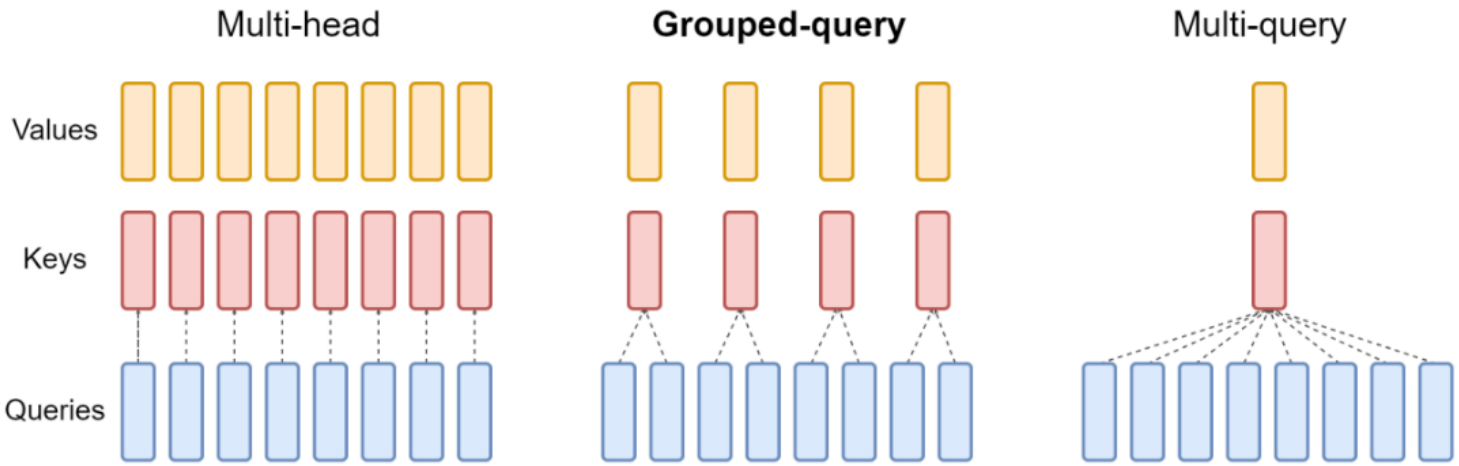
- 多个单头注意力组成MHA，使得模型能够在不同的表示子空间中同时关注序列中不同部分的信息，捕捉更多样化的依赖关系。

多查询注意力机制（MQA）

- 不同组共享Key向量和Value向量
  - Query可变
- 优势
  - 显存减少为原来的1/H
  - 推理可使用的Batch大小增加
  - 降低推理成本

群查询注意力机制（GQA）

- MHA和MQA之间的折中方案
  - 每组共享一个K、V向量，Query可变
- 优势
  - 显存消耗减小为MHA的G/H倍
  - 效果接近MHA，推理效率接近MQA



主要内容

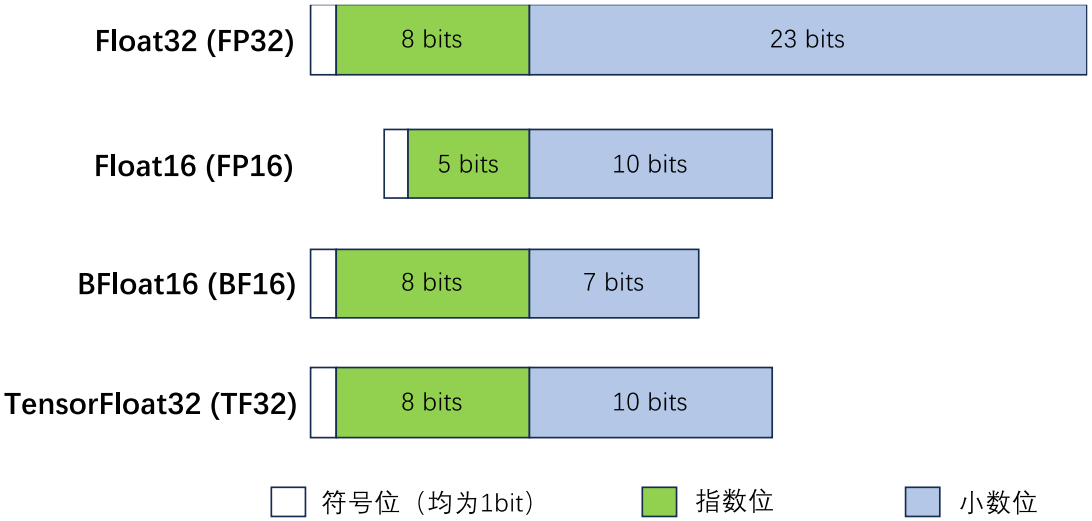
- 整体框架
- 预训练模型架构
- 分布式训练
- 预训练数据

分布式训练

- 基础优化技术
  - 混合精度训练
  - 算子加速方法
  - 梯度累加技术
  - 重计算技术
- 分布式训练技术
  - 数据并行
  - 参数切片并行
  - 张量并行
  - 流水线并行
  - 混合并行
- 训练参数设置

混合精度训练

- 使用不同精度数据类型进行训练
- 自动混合精度训练技术
  - 算子黑白名单搭配自动类型转换
  - 动态损失函数缩放（Loss Scaling）
  - 高精度基准权重（Master Weight）

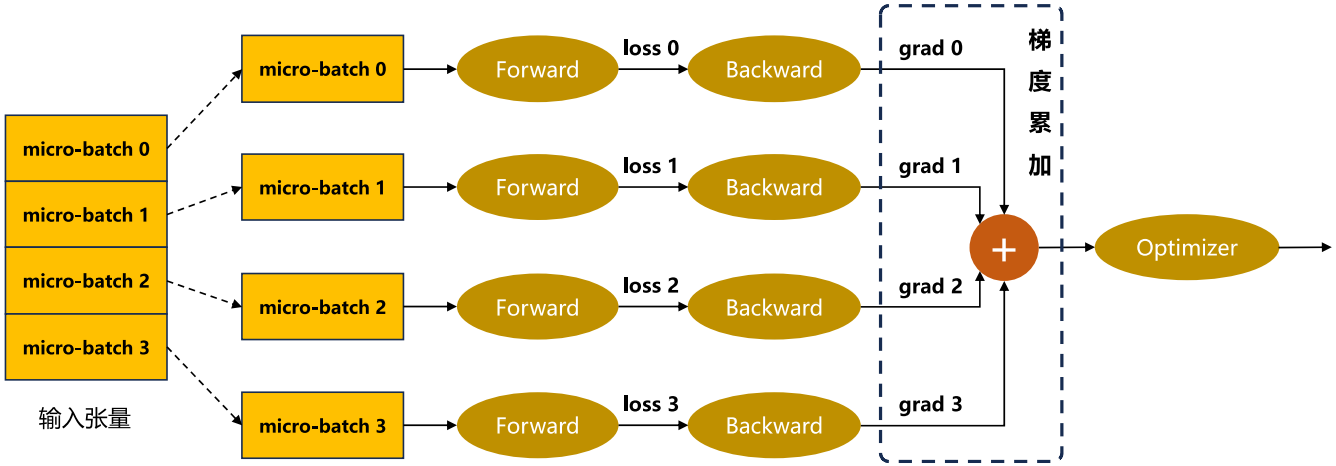


算子加速方法

- 方法1：实现更加高效的Kernel，提升每个单算子的速度
- 方法2：将多个算子融合为一个大算子，节省访存次数
  - FlashAttention技术
- 例：Llama大模型重点算子
  - 线性层（Linear）、旋转位置编码（Rotary Position Embedding，RoPE）、注意力（Attention）、激活层（SwiGLU）

梯度累加技术

- 批次大小（Batch size）受到硬件条件的限制
- 梯度累加技术
  - 将输入张量按Batch size维度拆分为多个微批次（micro batch）
  - 经过前反向计算，获得每个micro batch的参数梯度
  - 将参数梯度加和，进入优化器更新阶段
- 每个micro batch占用的显存较小，且在前向传播和反向传播后被释放，有效节省了模型激活的显存占用



重计算技术

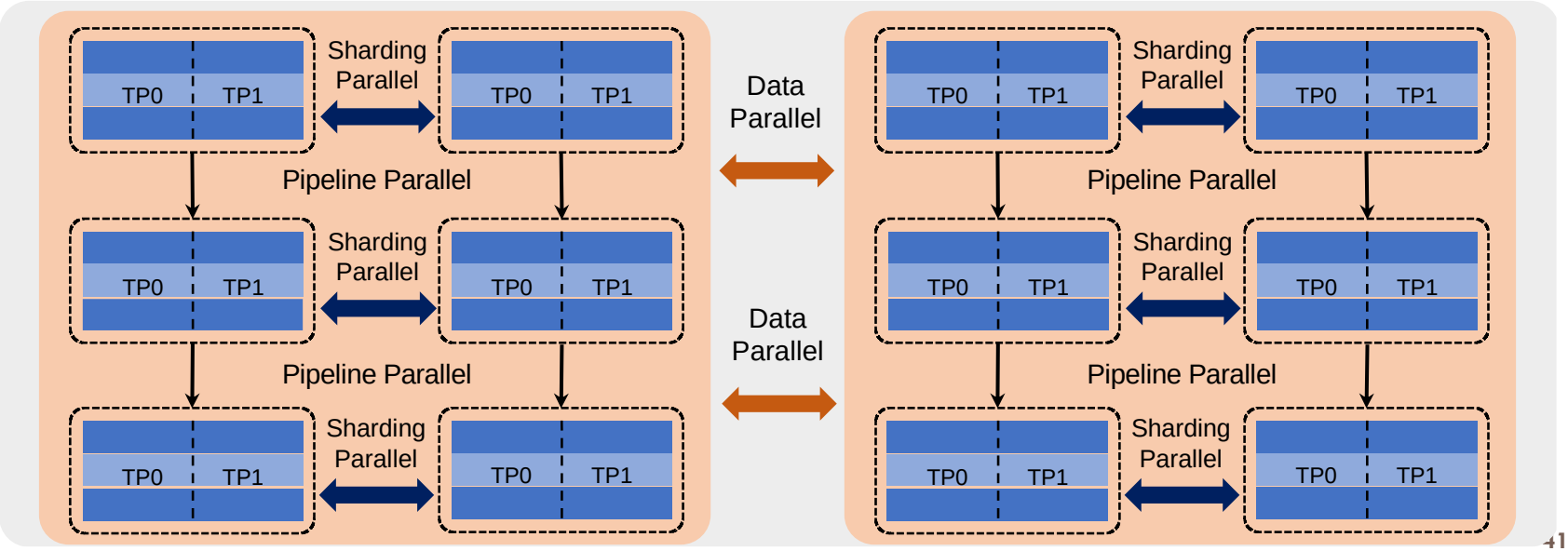
- 如果batch size=1也无法运行怎么办?
- 用时间换空间
  - 前向计算时舍弃反向计算依赖的前向激活显存，并在反向计算的时候重新计算一次前向算子，得到所需的前向激活
- 重计算策略可节省90%以上的激活显存占用
  - 但也带来了冗余计算，显存富裕的情况下需尽可能减少重计算

分布式训练

- 单卡GPU远远无法满足主流大模型的显存占用量
  - Llama2-70B 总显存达到1152GB
  - NVIDIA A100/A800单卡GPU显存80GB
- 如何将显存占用分摊到不同的设备上?
- 大模型分布式并行训练技术
  - 数据并行
  - 参数切片并行
  - 模型并行（张量并行、流水线并行）
  - 多维混合并行技术

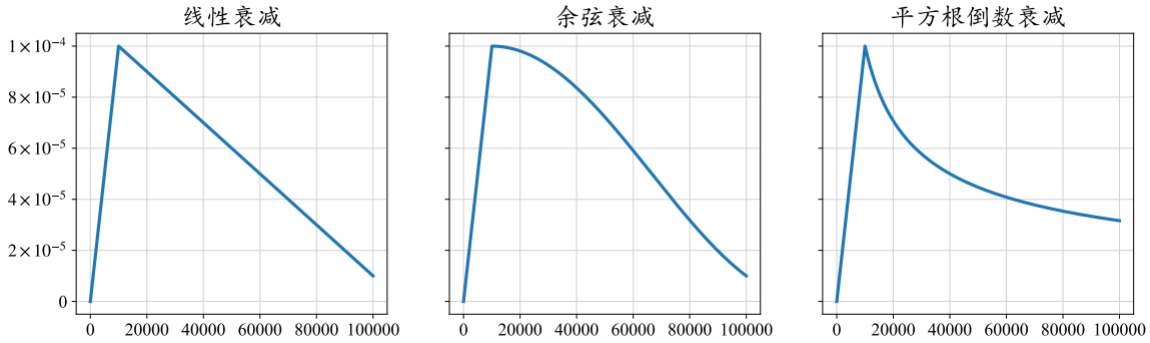
混合并行

- 取长补短
  - 节点内进行张量并行，节点间进行流水线并行
  - 使用参数切片并行节省Model State的显存
  - 通过数据并行提高整体的训练吞吐。



训练参数-学习率

- 控制模型在每次迭代中更新权重的步幅
  - 学习率高，训练过程快，但可能导致不稳定性和过拟合
  - 学习率低，稳定性强，训练时间长
- 调度策略动态调整
  - 预热阶段，用线性预热策略来逐步增大学习率，直到达到预先设定的最大值
    - 在学习率较高的情况下，模型的收敛速度会明显加快
  - 学习率达到最大值后逐步衰减，避免在最优解附近产生震荡
    - 衰减方法有线性衰减、余弦衰减和平方根倒数衰减等



训练参数-批次大小

- 每次训练迭代时使用的样本数量
  - 影响模型的收敛速度、训练稳定性
- 动态批次调整策略
  - 大规模训练中可以有效稳定训练过程
    - GPT-3批次大小32K->3.2M个词元
  - 主要思想
    - 早期较小批次提升反向传播的频率，让模型损失尽快下降
    - 后期较大批次使损失下降更加稳定，模型更好收敛
  - 需动态调整学习率

训练参数-训练轮数

- 模型在训练集上完整学习多少遍
- 调整策略
  - 对小数据集增加训练轮数，让模型充分拟合数据
  - 对大规模数据集减少训练轮数，防止过拟合

训练参数-优化器

- 调整模型的训练参数
- 常用的优化器
  - Adam（Adaptive Moment Estimation）
    - 减少梯度更新过程中的震荡
  - AdamW
    - 引入权重衰减（L2正则化）防止过拟合
  - Adafactor
    - 每个更新步骤中只存储必要的信息来节省显存，适用于资源受限的场景

主要内容

- 整体框架
- 预训练模型架构
- 分布式训练
- 预训练数据

常见数据来源：语言文本语料库以外

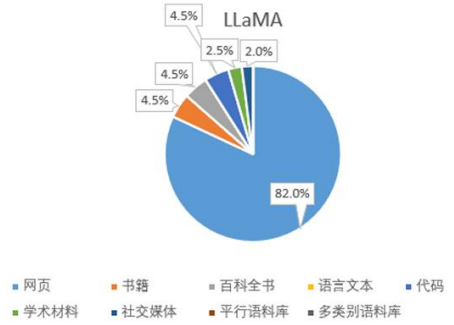
来源	特点	代表性数据集
网页	最大规模，需清洗	Common Crawl, C4
书籍	高质量长文本	Project Gutenberg
百科	知识权威、结构规范	Wikipedia
代码	程序语言	The Stack
科研文献	专业前沿	arXiv
社交媒体	实时互动性	Reddit
平行语料	多语言对齐	ParaCrawl, MultiUN
多模态转文字	口语化、场景化	视频字幕、语音对话

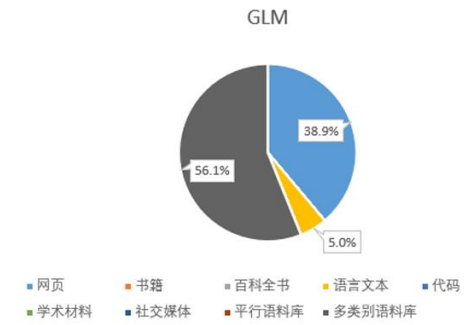
数据配比

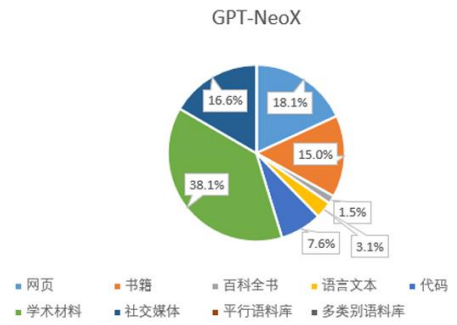
- 数据配比目标
  - 多样性，学习丰富特征
  - 平衡性，避免对某一类数据产生过度依赖
  - 针对性，针对目标任务选择相关数据
- 常用配比方法
  - 基于数据分布的数据配比
    - 贴合真实世界数据的分布，优化训练数据的构成
  - 基于数据质量的数据配比
    - 优先选择高质量数据
  - 基于模型性能的数据配比
    - 观察验证集上的效果调整配比

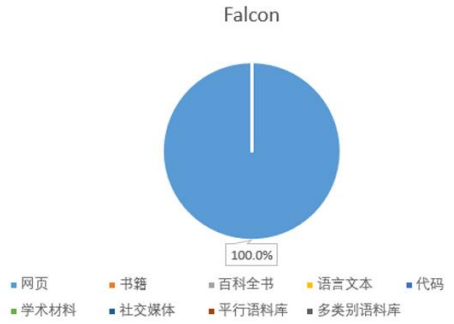
数据配比

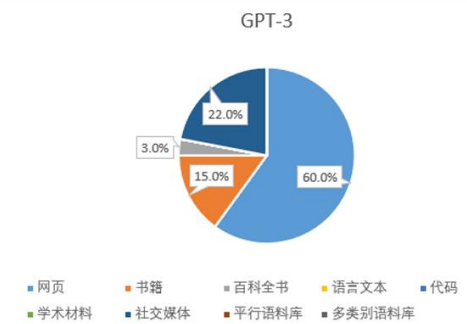
- 下图展示了不同大语言模型使用的预训练语料库中的数据类型分布。

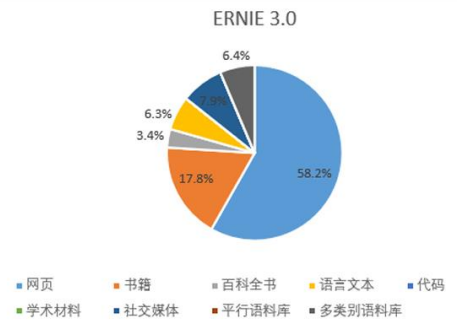












数据筛选：补充材料

- 数据过滤：通过基于模型或启发式规则的方法提升数据质量，包括训练分类器过滤有害内容，以及在文档级别和句子级别实施规则过滤，确保筛选出高质量内容。
- 数据去重：利用MinHash等高效算法识别并移除重复或高度相似的文本，确保数据多样性。
- 数据标准化：对文本进行系统化转换，包含句子切分、拼写校正和停用词等环节，将原始文本转化为结构化格式，使数据更适合模型训练和理解。
- 数据审查：建立完整的处理档案并进行抽样检查，通过手动评估数据质量发现问题，确保预训练数据的可靠性与适用性，为模型训练提供坚实基础。

数据合成：补充材料

- 高质量数据是训练优秀大语言模型的基础。然而，高质量数据的增长速度远远落后于训练数据集的扩张速度，导致了数据枯竭危机的出现。这凸显了探索新数据来源的迫切需求，数据合成是一种解决当前困境的方案。

- 通常，真实的数据是直接从现实世界中获取的，而数据合成的目标是通过算法、模拟或者生成式模型构建的人工数据，让这些数据和真实数据有着极高的相似程度。

结语：大语言模型预训练

- 主流模型架构：Transformer
  - 位置编码
  - 层归一化
  - 注意力
- 分布式训练
  - 显存
  - 时间
- 数据
  - 来源
  - 配比